

ENGINEERING BLACK BOOK

RESDEV AI

From offline-first prototype to a tested public AI resume automation system.

Author	Kanishk Surwade
Project	ResDev AI — Evidence-Grounded Resume Automation
Public Demo	resdev-ai.streamlit.app
Repository	github.com/KanishkSurwade/ResDev-AI
Final Verified Run	ATS 94/100 • Semantic 92/100 • Combined 93/100 • Multi-ATS 6/6
Automated Tests	136 passed • 1 external-library deprecation warning

Purpose of this book. This document records the engineering journey, architecture evolution, implementation milestones, failures, debugging, production hardening, testing and final product behavior. It is intended as a portfolio artifact and as a future engineering reference.

Table of Contents

1	Executive Summary
2	Original Problem
3	Original Vision
4	Initial Architecture — Offline First
5	Foundation and Master Resume
6	Job Description Analysis
7	JD ↔ Master Resume Matching
8	Resume Generation
9	Resume Quality Evaluation
10	Automatic Optimization Loop
11	Deterministic ATS Analyzer
12	Multi-ATS Compatibility
13	Evidence-Grounded ATS Optimization
14	Structured Validation
15	Streamlit Frontend and PDF Rendering
16	Major Architecture Change — Local to Cloud
17	Secure Cloud Configuration
18	Cloud Deployment Import Problems
19	Ollama localhost Failure
20	LaTeX / PDF Cloud Failure
21	Streamlit Stopping / Connection Behavior
22	Multi-ATS Test Regression
23	Optimization Bottlenecks
24	Production Verification
25	Demo Evidence
26	Final Architecture
27	Final User Workflow
28	Testing Strategy
29	Engineering Decisions
30	Failure Catalogue
31	What the Project Demonstrates
32	Lessons Learned
33	Final Status
34	Future Roadmap
35	Final Reflection
36	Visual Evidence
Appendix A	Repository Milestones
Appendix B	Final Evidence Snapshot

1. Executive Summary

ResDev AI automates resume customization from a Job Title, Job Description, Master Resume and Resume Template. The system analyzes the job, matches candidate evidence, generates a targeted resume, evaluates quality, iteratively improves the candidate and produces structured JSON, LaTeX and PDF outputs.

- The master resume is the source of truth for candidate facts.
- The pipeline separates probabilistic AI generation from deterministic validation.
- The public workflow uses a cloud-compatible Gemini path while the local workflow retains the offline-first Ollama + Qwen path.
- The final verified state included 136 passing automated tests, ATS 94/100, Semantic 92/100, Combined 93/100 and Multi-ATS 6/6.

2. Original Problem

Manual resume tailoring repeats the same work for every application: reading the job description, extracting requirements, locating relevant experience, rewriting bullets, checking keywords and structure, formatting the document and exporting a PDF. ResDev AI was designed to collapse these repeated steps into one workflow.

- Reduce repetitive manual tailoring.
- Keep job-specific relevance without creating unsupported claims.
- Make evaluation measurable rather than relying on whether an LLM response merely looks good.

3. Original Vision

The intended user experience was deliberately small: the user supplies four inputs and the system performs the remaining work automatically.

- Job Title
- Job Description
- Master Resume
- Resume Template

4. Initial Architecture — Offline First

The original technical direction favored free, local and open-source components: Python, Streamlit, Ollama, Qwen, JSON, SQLite, LaTeX, Git and GitHub. The local architecture kept the AI runtime on the developer machine and aimed to work offline whenever possible.

- Local AI runtime: Ollama + Qwen.
- Structured data: JSON, with SQLite intended for local history/storage components.
- Document generation: LaTeX followed by PDF compilation.
- Version control and portfolio publication: Git + GitHub.

5. Foundation and Master Resume

The system first established structured master-resume data and validation. This was a critical architectural boundary: downstream AI components could select and adapt evidence, but they were not allowed to manufacture candidate facts.

- Candidate information, capabilities, experience, projects, education and certifications were represented structurally.
- Validation protected required sections and fields.
- Generation rules explicitly disallowed fabrication of employers, roles, skills, projects, achievements, metrics and technologies.

6. Job Description Analysis

JD analysis became the first intelligence layer. The system extracts requirements that can later drive matching and generation.

- Required and preferred skills.
- Keywords.
- Responsibilities.
- Tools and technologies.
- Experience signals and other role requirements.

7. JD ↔ Master Resume Matching

Matching connected job requirements with candidate evidence. The design distinguished supported evidence, transferable evidence and missing requirements instead of pretending every JD requirement was satisfied.

- Strong match: explicit evidence exists.
- Transferable match: closely related evidence can support the requirement.
- Missing: no reasonable evidence exists in the Master Resume.
- The matching score is a heuristic semantic measure, not a universal ATS score.

8. Resume Generation

Generation was built on top of JD analysis and matching. The model receives structured candidate evidence, structured job information and alignment context, then produces a structured targeted resume.

- The output schema covers personal information, summary, skills, experience, projects, education, certifications and tailoring metadata.
- The generator is instructed to use strong action verbs and natural keyword alignment only where supported.
- The downstream system validates and normalizes the structured output before rendering.

9. Resume Quality Evaluation

A dedicated evaluation layer was added so the first generated resume did not automatically become the final resume.

- Keyword Match

- Skills Match
- Experience Relevance
- Grammar
- Formatting
- ATS Structure
- Readability
- Resume Length
- Strong Action Verbs
- Missing Keywords

10. Automatic Optimization Loop

The architecture evolved from one-shot generation to an iterative loop: generate → evaluate → create a targeted edit/revision plan → regenerate where useful → evaluate again → keep the strongest candidate.

- The loop can reject a candidate when it does not meaningfully improve the current best.
- An optimization guard protects against regressions.
- Iteration limits prevent uncontrolled generation.
- The objective is measurable improvement rather than simply producing more text.

11. Deterministic ATS Analyzer

A deterministic ATS analyzer was introduced so important structural and keyword checks were not dependent entirely on an LLM judgment. This improved repeatability and made regression testing practical.

- Deterministic checks complement qualitative semantic evaluation.
- ATS weaknesses become actionable signals for revision.
- The system can report an internal ATS score without claiming access to commercial ATS internals.

12. Multi-ATS Compatibility

A Multi-ATS validator was added to simulate compatibility across multiple ATS-style profiles. The purpose is transparent internal compatibility testing rather than claiming official access to commercial ATS systems.

- The validator exposes structural weaknesses that a single score can hide.
- The final verified run passed 6/6 internal Multi-ATS profiles.
- The metric is an internal engineering signal, not a guarantee of commercial ATS behavior.

13. Evidence-Grounded ATS Optimization

The architecture was consolidated around evidence-grounded optimization. Valid JD gaps can be improved when the Master Resume contains supporting evidence; unsupported requirements remain visible rather than being fabricated.

- The system prioritizes relevant evidence.
- Unsupported requirements are surfaced through an honest audit.
- Optimization is constrained by factual truth.

14. Structured Validation

Final structured-resume validation protects the boundary between AI output and deterministic downstream processing. This matters because malformed JSON can break rendering even when the generated prose appears correct.

- Validate required top-level sections.
- Normalize missing or malformed fields.
- Protect the LaTeX/PDF contract from invalid structured data.

15. Streamlit Frontend and PDF Rendering

A production-facing Streamlit interface was introduced. It exposes the input workflow, optimization progress, iteration log, results and downloads. LaTeX is the document-generation layer for professional PDF output.

- Input levels: Job Information, Master Resume and Resume Template.
- Optimization Pipeline with iteration log.
- Results with ATS, Semantic, Combined and Multi-ATS metrics.
- Downloads for PDF, LaTeX and JSON.

16. Major Architecture Change — Local to Cloud

The largest architecture change occurred when the application moved from local execution to public hosting. Ollama worked on the developer machine, but a hosted Streamlit container could not access the developer's localhost. The public path therefore migrated to Gemini while preserving the core pipeline.

- Local mode: Streamlit → Python pipeline → Ollama + Qwen → local outputs.
- Public mode: Streamlit Cloud → Python pipeline → Gemini → LaTeX/PDF → downloads.
- The application architecture remained shared; only the AI runtime changed by environment.

17. Secure Cloud Configuration

Cloud migration required secrets to be separated from source code. Gemini credentials were moved toward environment variables / Streamlit secrets so public repository code did not contain private credentials.

- Never commit API keys, passwords, access tokens or private credentials.
- Cloud configuration belongs in deployment secrets.
- Local and cloud execution paths remain explicit.

18. Cloud Deployment Import Problems

The hosted environment exposed package and import differences that were not visible locally. The deployment was repeatedly tested and dependency/import handling was corrected instead of assuming local success implied cloud success.

- Local and hosted Python environments are not identical.
- Deployment dependencies must be declared explicitly.
- Production verification must exercise the hosted path itself.

19. Ollama localhost Failure

One of the clearest failures was a cloud attempt to reach **http://localhost:11434** for Ollama. In a hosted container, localhost refers to the container itself, not the developer's laptop. The failure was architectural, not a bad prompt or resume.

- Observed symptom: connection failure to the Ollama generation endpoint.
- Root cause: the hosted container had no Ollama service at its own localhost.
- Correct fix: use a cloud-compatible model provider for the hosted path while retaining Ollama locally.

20. LaTeX / PDF Cloud Failure

The hosted system initially generated the LaTeX source but could not compile the PDF because the cloud environment did not contain the required LaTeX tooling. The application was hardened to detect the situation honestly, and the deployment was updated with system package configuration.

- Separate LaTeX generation from PDF compilation.
- Detect missing pdflatex/tooling rather than crashing while opening a nonexistent PDF.
- Declare system dependencies needed by the deployment.
- Verify the actual PDF download path in the hosted environment.

21. Streamlit Stopping / Connection Behavior

During local testing, Streamlit sometimes appeared to remain in a stopping state after an interrupted process. Terminal output showed connection-reset behavior. The issue was treated as runtime/process behavior rather than a reason to redesign the pipeline.

- Stop and restart the Streamlit process cleanly.
- Separate runtime shutdown symptoms from application logic failures.
- Use the automated test suite to validate the code independently of the UI process.

22. Multi-ATS Test Regression

A regression test exposed a case where a rich fixture unexpectedly produced 0/6 expected Multi-ATS passes. The quality requirement was not weakened. The validator/test behavior was corrected until the intended fixture behavior was restored.

- Initial regression: 0/6 on a fixture explicitly designed to satisfy multiple profiles.

- Fix direction: correct validator/test behavior rather than lowering the acceptance requirement.
- Final suite after correction: 136 passed.

23. Optimization Bottlenecks

Several runs produced strong semantic scores while ATS scores remained weaker. Other runs reached MAX_ITERATIONS_REACHED without reaching the desired combined target. These failures showed why deterministic signals, revision planning and regression protection were necessary.

- Semantic quality alone does not guarantee ATS compatibility.
- More generations are not automatically better.
- The optimizer needs a measurable acceptance rule and rollback protection.
- The strongest verified run ultimately reached ATS 94, Semantic 92 and Combined 93.

24. Production Verification

Production verification included both local automated tests and actual public end-to-end usage. The final local test run reported 136 passing tests with one external-library deprecation warning.

- Local automated tests: 136 passed.
- External warning: Google GenAI dependency deprecation warning; it did not fail the suite.
- Public flow exercised: input → optimization → results → downloads.
- Strongest verified result: ATS 94/100, Semantic 92/100, Combined 93/100, Multi-ATS 6/6.

25. Demo Evidence

The recorded product demonstration captures the actual interface and end-to-end workflow: landing page, job input, master resume upload, template selection, optimization pipeline, iteration log, results and downloadable artifacts. The demo represents an earlier development state; later engineering iterations improved the verified metrics.

- Demo duration recorded in the Black Book evidence: 111.9 seconds.
- Historical demo evidence is retained because it shows the product evolution rather than only the final state.

26. Final Architecture

The final high-level architecture is:

```
User
↓
Streamlit UI
↓
JD Analyzer ████████ Resume Parser / Matcher
\ /
↓
Evidence-Grounded Generator
↓
Quality Evaluator
↓
```



```
Deterministic ATS Analyzer
↓
Multi-ATS Validator
↓
Optimization Guard / Revision Loop
↓
Structured Resume
■ ↓ ■
JSON LaTeX PDF
```

Runtime split: local mode can use the offline-first Ollama path; public cloud mode uses a cloud-compatible Gemini path.

27. Final User Workflow

The final user experience is intentionally minimal:

- 1. Enter Job Title.
- 2. Paste Job Description.
- 3. Upload Master Resume.
- 4. Select Resume Template.
- 5. Click Generate Resume.
- 6. Wait for automated analysis and optimization.
- 7. Download PDF, LaTeX and JSON.

28. Testing Strategy

Testing was treated as a first-class engineering activity. Regression tests cover ATS behavior, optimization guards, production/cloud pipeline behavior and Multi-ATS validation.

- The final suite reached 136 passing tests.
- Tests protect behavior that should remain stable as AI logic evolves.
- The test suite was used to resolve regressions rather than hiding them.

29. Engineering Decisions

The project deliberately favored a simple modular design instead of adding unnecessary infrastructure.

- Prefer simple Python modules over unnecessary technologies.
- Keep the Master Resume as the evidence boundary.
- Use deterministic evaluation around probabilistic LLM output.
- Use optimization rather than one-shot generation.
- Use Multi-ATS simulation instead of claiming a universal ATS API.
- Keep local/offline capability where practical.
- Use Gemini only where public hosting requires a cloud-compatible model.
- Keep secrets outside source code.

- Treat failures as engineering evidence and fix root causes.

30. What Went Wrong — Failure Catalogue

The project encountered real bottlenecks across development and deployment. Each failure became an engineering input rather than being hidden.

- Local/cloud runtime differences.
- Ollama localhost assumptions.
- Cloud package/import differences.
- Missing LaTeX tooling.
- PDF path handling.
- Streamlit process shutdown behavior.
- Multi-ATS regression.
- Weak ATS scores despite high semantic scores.
- Iteration limits and MAX_ITERATIONS_REACHED states.

31. What This Project Demonstrates

ResDev AI demonstrates practical work across the full software lifecycle:

- Python development and modular architecture.
- AI/LLM integration and prompt engineering.
- Structured JSON data handling.
- Deterministic evaluation and ATS-style validation.
- Iterative optimization and regression protection.
- LaTeX rendering and PDF generation.
- Streamlit UI development.
- Cloud configuration and deployment.
- Automated testing.
- Git/GitHub workflow.
- Debugging a real production-style application.

32. Lessons Learned

The largest lesson is that a working AI prompt is not the same thing as a production system. Reliability required validation, deterministic checks, regression tests, error handling, environment-aware deployment, secure configuration, PDF tooling and repeatable verification.

- Build one feature at a time.
- Use failures to guide architecture.
- Keep probabilistic AI components surrounded by deterministic contracts.

- Verify local and cloud environments separately.
- Do not confuse internal quality metrics with guarantees about external systems.

33. Final Status

ResDev AI reached a working public-demo state with the complete generation workflow, downloadable outputs, automated optimization, ATS evaluation, Multi-ATS validation, cloud AI configuration, PDF rendering support, documentation and automated tests. The repository is public and the Streamlit demo is live.

- Public demo: resdev-ai.streamlit.app
- Repository: github.com/KanishkSurwade/ResDev-AI
- Outputs: PDF, LaTeX and JSON
- Tests: 136 passed
- Best verified metrics: ATS 94, Semantic 92, Combined 93, Multi-ATS 6/6

34. Future Roadmap

Future improvements were intentionally kept separate from the production milestone.

- More resume templates.
- Resume history.
- Optional Google Drive / Sheets integrations.
- Additional evaluation methods.
- Stronger privacy controls.
- More deployment options.

35. Final Reflection

ResDev AI was built as a learning-by-building project. The final result is not only a resume generator; it is a record of practical software engineering: architecture, AI integration, deterministic validation, testing, debugging, cloud deployment and production hardening.

- The project converted a personal workflow problem into a modular application.
- The architecture evolved because real deployment constraints exposed weaknesses.
- The Black Book preserves those decisions and failures so the project can be understood and extended later.

36. Visual Evidence from the Product Demo

The product evidence package includes screenshots of the landing page, input sections, template selection, optimization pipeline, iteration log, results and downloadable artifacts. These visuals document the user-facing product and complement the engineering narrative.

- Landing page / product identity.
- Job title and job description input.

- Master resume upload and successful parsing.
- Resume template selection.
- Optimization pipeline and iteration log.
- Final results with ATS, semantic, combined and Multi-ATS metrics.
- PDF, LaTeX and JSON download controls.

Appendix A — Repository Milestones

The Git history provides an auditable implementation timeline. The recorded milestones form a practical development chronology:

- 01. Project foundation
- 02. Master resume data and validation
- 03. JD analysis
- 04. JD/master-resume matching
- 05. Resume generation
- 06. Resume quality evaluation
- 07. Automatic optimization
- 08. Deterministic ATS analysis
- 09. Multi-ATS validation
- 10. Evidence-grounded ATS optimization
- 11. Structured validation
- 12. Streamlit frontend and PDF rendering
- 13. Gemini cloud migration
- 14. Secure Gemini configuration
- 15. Deployment import fixes
- 16. Resume upload/parsing
- 17. Optimization/evaluation improvements
- 18. Production pipeline finalization
- 19. Documentation and cloud pipeline tests
- 20. Final Streamlit Cloud PDF-generation fix

Appendix B — Final Evidence Snapshot

Artifact / Metric	Verified State
Automated tests	136 passed; 1 external-library deprecation warning
Best verified ATS	94 / 100
Semantic score	92 / 100
Combined score	93 / 100
Multi-ATS	6 / 6 passed
Outputs	PDF • LaTeX (.tex) • JSON
Public demo	resdev-ai.streamlit.app
Repository	github.com/KanishkSurwade/ResDev-AI

Important metric note: ATS and Multi-ATS values are internal evaluation metrics. They are not official scores from commercial ATS vendors and should not be presented as guarantees.

End of Engineering Black Book

ResDev AI — Evidence-Grounded Resume Automation • Built by Kanishk Surwade